Look-Ahead EDF

The idea is that we'll defer as much work as possible and set the CPU frequency as low as possible.

If, at a later stage, a task uses less CPU than worst case, then deferred work may never be needed.

Ensures there are sufficient cycles available for each task to meet its deadline after reserving cycles for high priority jobs.

Look-Ahead EDF

"Reserve clock" by deferring tasks if they can be run after the current task's deadline.

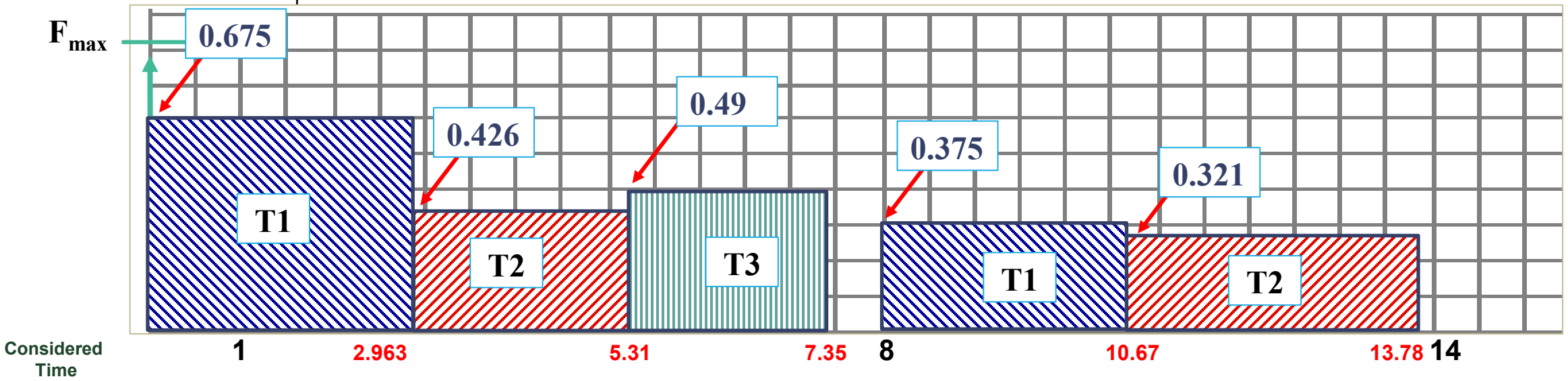
For example, lets consider the same set of tasks from before:

Task	Period	Execution Time (Worst Case)	Execution Time 1 (Actual)	Execution Time 2 (Actual)
T_1	8	3	2	1
T_2	10	3	1	1
T_3	14	1	1	1

For this exercise, each task only runs the 2 times.

Look-Ahead EDF

Here's the same example, but without p-states (all frequencies available). We can observe that look-ahead EDF allows us to fill in the regions between T_3 and T_1 that would otherwise be idle using the simpler cycle conserving EDF:



T1	T1 (2nd release) Def	T1 (2nd release)	T1 (2nd release)	
T2	T2	T2 (2nd release) Def	T2 (2nd release) Def	T2
T3 Deferred	T3 Def	T3	T3 (2nd release) Def	T3 (2nd release) Def

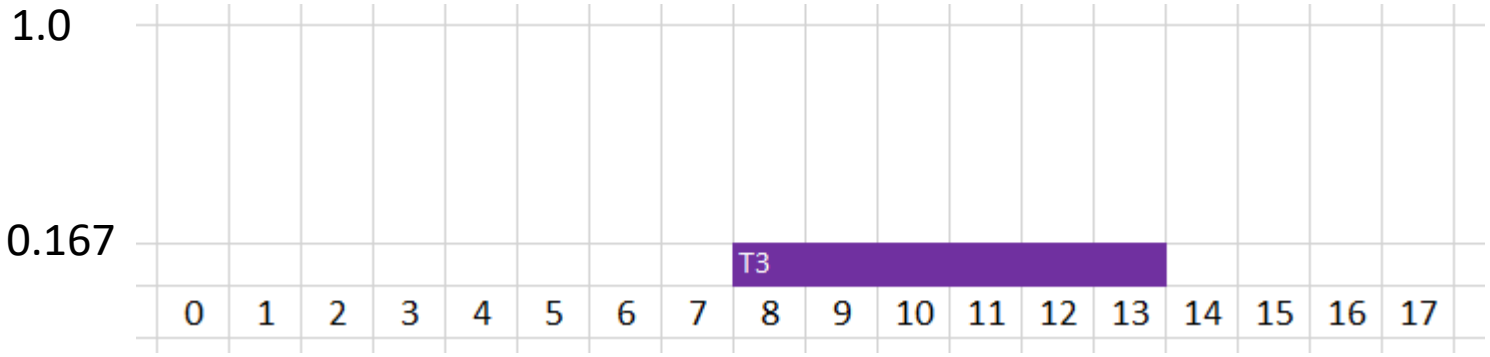
The blue tasks considered for scheduling, the brown are deferred tasks.

Question

Can we do even better?

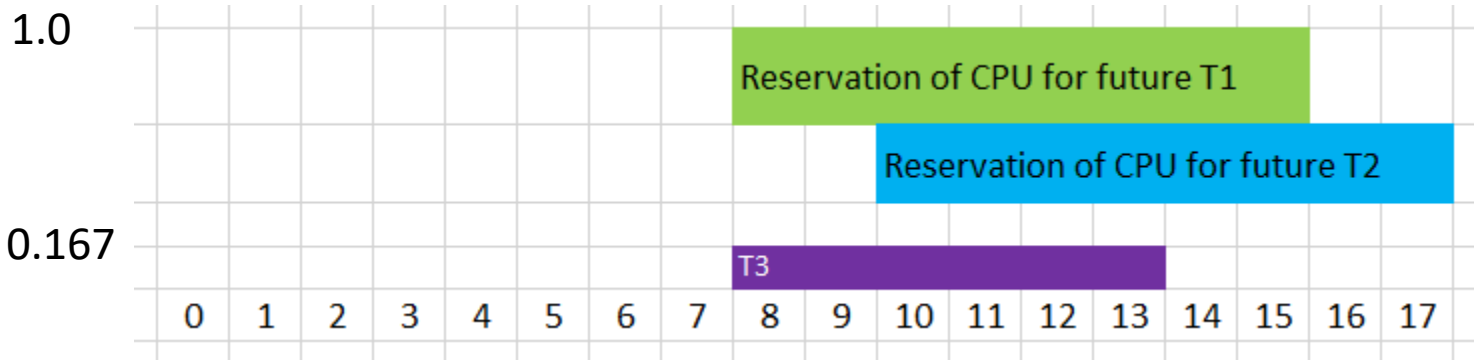
Look-Ahead EDF

Considering the same tasks as before, let's plan to defer T_3 until after the first deadline in our set of tasks (completion of T_1) and spread its work across that time interval.



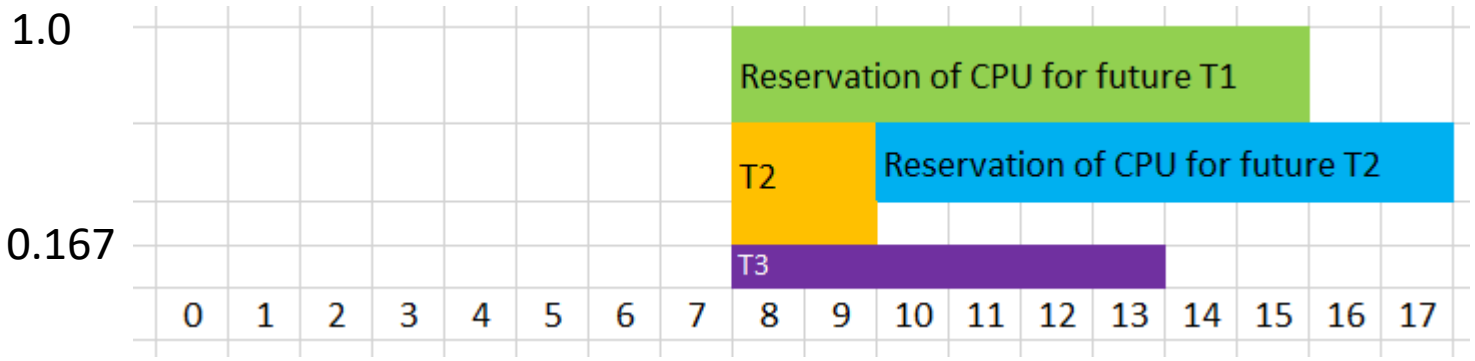
Look-Ahead EDF

Now that we can guarantee completion of T_3 by its deadline (we have enough area in the plot), lets reserve time for a future T_1 and T_2. The future T_1 runs from 8 to 15, requiring the worst case area of 3. The future T_2 runs from 10 to 17, requiring the worst case area of 3 as well.



Look-Ahead EDF

Now we can schedule as much of T2 as possible in the remaining area in time units 8 and 9:

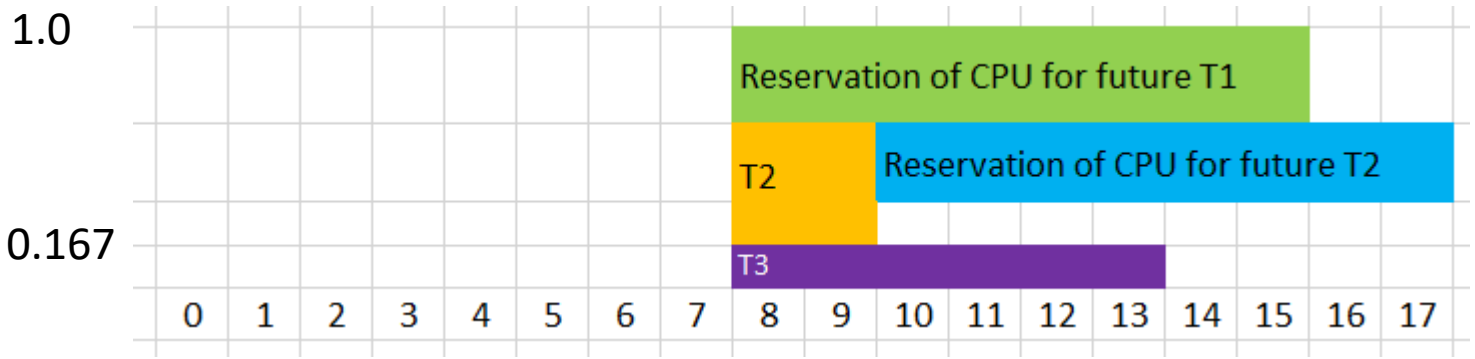


Look-Ahead EDF

T_2 requires a worst-case area of 3, but we were only able to guarantee an execution time of 0.9167. 0.9167 is the area of T_2, which can be computed as:

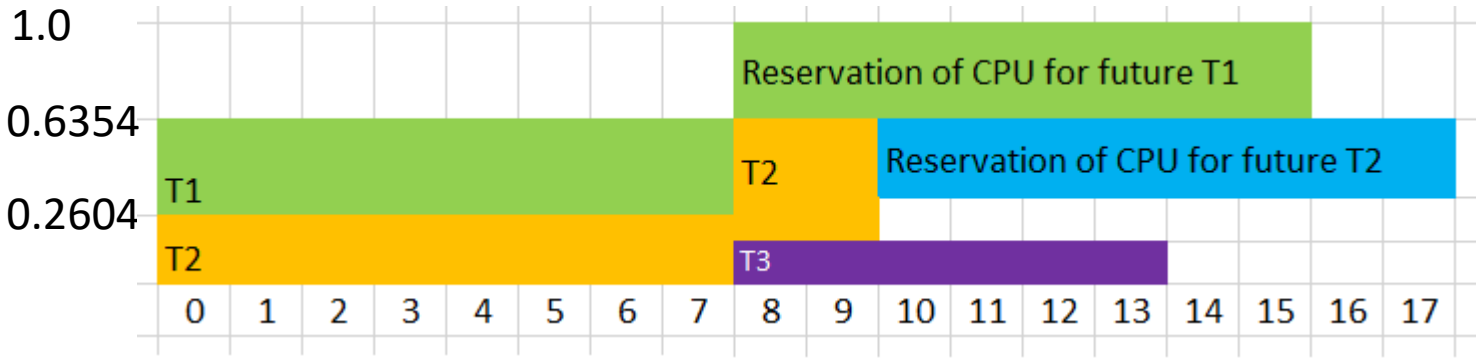
2 time units multiplied by the leftover CPU clock: 2*(1-3/8-1/6).

This leaves 2.0833 as the execution time for T_2 that must be scheduled over the first period of T_1.



Look-Ahead EDF

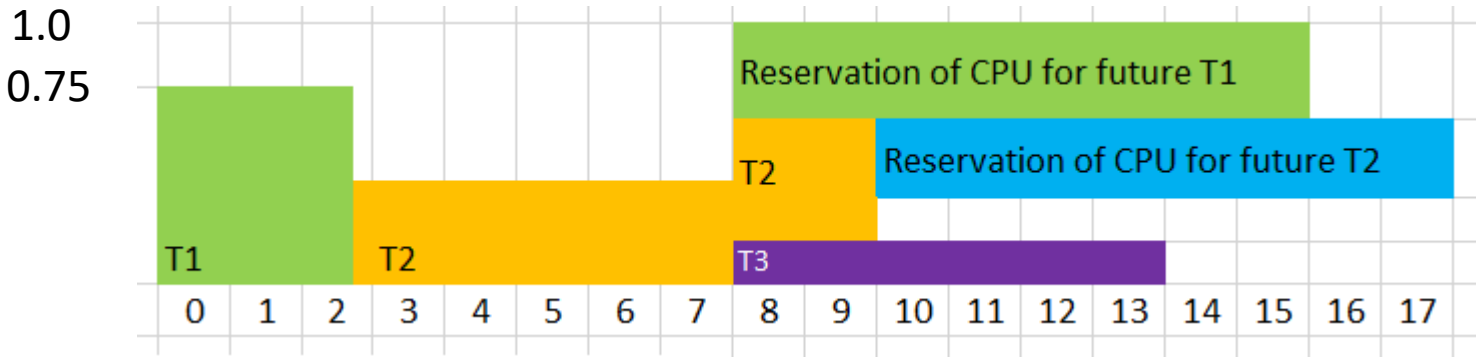
Now we add T_1 and T_2 and their required area to the first period of T_1.



Look-Ahead EDF

Now we can pick a frequency to run the clock at. Typically we will be limited to p-states, so lets assume we have p-states of 1, 0.75, 0.5 and 0.25 available to us.

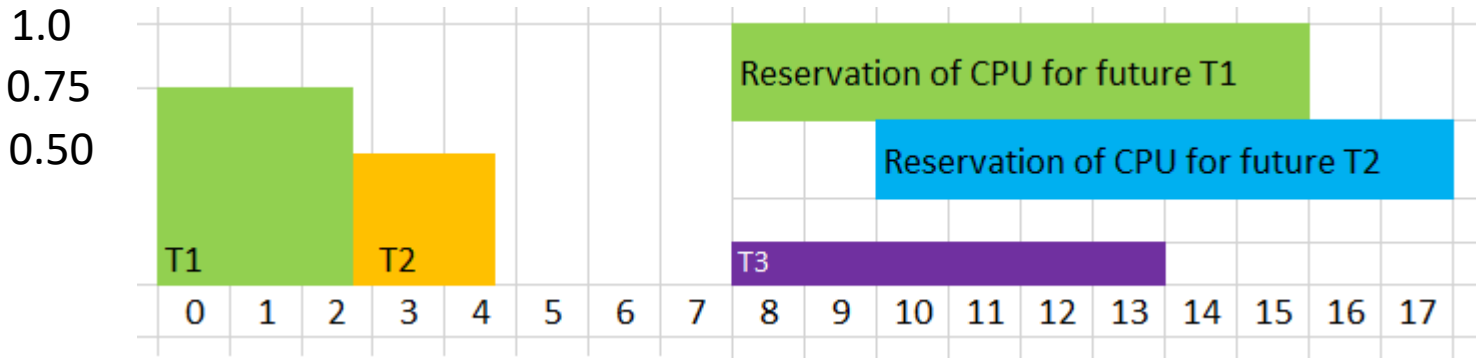
We find a K value such that, as reserved, T_1 and T_2 can complete the necessary work by 8. K=1/0.675. We must then jump to the next available p-state of 0.75. We then run T_1 and find T_1 finished early, taking only 2 execution time instead of 3. This corresponds to time 2/0.75=2.67:



Look-Ahead EDF

This gives additional time to run T2, so we calculate a new K value to ensure that T2 still completes before its deadline. We get a value of 1/0.409, so we jump to the next closest p-state and run at a frequency of 0.5.

T2 completes early, taking only 1 execution time instead of the expected 3. That places us at 2.67+1/0.5=4.67



Look-Ahead EDF

We technically don't have T_3 set to run until time 8, but EDF is work conserving, we run tasks when we can. So, launch T_3 at time 4.66 with a K=1/0.482 (reservation for T_1 included, but T_2 is deferred). We bump to the closest p-state, so we will run T_3 at a frequency of 0.5, allowing T3 to complete by 4.67+1/0.5=6.67: